

Step 2 — Parallel Lookups With `RunnableParallel` — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real LCEL calls), **Advanced** (proving the concurrency actually happened, with real numbers). Read Basic first even if you've used `RunnableParallel` before — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Why two independent lookups belong in `RunnableParallel`](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Why two independent lookups belong in `RunnableParallel`

Basic Version

If two lookups don't need each other's answer to run — the account lookup doesn't need the order history, and the order history doesn't need the account info — there's no reason to make one wait for the other to finish. `RunnableParallel` runs several things at once and hands you back one dict with all their results in it.

Things to use: - Two plain Python functions that each take a bit of time (pretend they're calling a real service). - `RunnableParallel` from `langchain_core.runnables` . - `RunnableLambda` to wrap a plain function so it can sit inside a chain.

Intermediate Version

`RunnableParallel(name1=chain1, name2=chain2, ...)` runs every value concurrently and returns a dict shaped exactly like the keyword arguments you gave it — `{"name1": result1, "name2": result2}` . A plain Python function isn't automatically a `Runnable` , so wrap it first: `RunnableLambda(get_account_info)` . `LangChain` runs the wrapped, synchronous functions in a background thread pool, so a `time.sleep(1)` inside one of them doesn't block the other one from also running during that same second.

`RunnablePassthrough()` is the piece that says "just hand the original input straight through, unchanged" — useful here so the ticket text itself survives alongside the two lookup results, instead of getting lost once `RunnableParallel` takes over.

The exact pieces: - `from langchain_core.runnables import RunnableParallel, RunnableLambda, RunnablePassthrough` - `import time` for the fake network delay. - `RunnableParallel(account_info=RunnableLambda(get_account_info), order_history=RunnableLambda(get_order_history), ticket=RunnablePassthrough())` - `time.perf_counter()` (not `time.time()`) for measuring how long something actually took — it's the right tool for timing code, not for telling wall-clock time.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

"I used `RunnableParallel`, so it must be running in parallel" is a claim, not proof. The only way to actually know is to measure it: call both lookup functions one after another by hand and time that, then call them through `RunnableParallel` and time that. If both lookups sleep for about 1 second, the sequential version should take about 2 seconds total, and the parallel version should take about 1 second — because both sleeps happen at the same time, in different threads, instead of one after the other.

If your "parallel" timing comes out close to 2 seconds anyway, something is wrong — the most common cause is accidentally calling both functions directly (not through `RunnableLambda`, not through `RunnableParallel`) somewhere else in your code, so the `RunnableParallel` version never actually gets exercised.

Think about it yourself before Hint 2: what would you actually print, and what numbers would convince a skeptical reviewer that this is really running in parallel, not just labeled "parallel" in a variable name?

Difference between Basic, Intermediate, and Advanced: Basic explains why independent lookups shouldn't be sequential. Intermediate gives the real `RunnableParallel` + `RunnableLambda` + `RunnablePassthrough` calls. Advanced is about proof, not new API — timing both versions with real numbers is the only way to know your "parallel" code is actually behaving differently from sequential code, instead of just looking different on the page.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
function get_account_info(inputs):
    pretend this is a slow database call (sleep 1 second)
    return made-up account data

function get_order_history(inputs):
    pretend this is a slow API call (sleep 1 second)
    return made-up order data

build a RunnableParallel that runs both at once, plus keeps the original ticket

time: call both one after another -> should take about 2 seconds
time: call both through RunnableParallel -> should take about 1 second
```

Intermediate Version

```
# lookups.py
import time

def get_account_info(inputs):
    # stand-in for a real database or CRM API call
    time.sleep(1)
    return {"plan": "Pro", "customer_since": "2022-01-01"}

def get_order_history(inputs):
    # stand-in for a real orders-service API call
    time.sleep(1)
    return {"recent_orders": ["INV-2201", "INV-2150"]}
```

Now write `research.py`'s `build_research_chain()` yourself, and the sequential-vs-parallel timing script, before checking the Advanced version below.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
# research.py
from langchain_core.runnables import RunnableParallel, RunnableLambda, RunnablePassthrough
from lookups import get_account_info, get_order_history

def build_research_chain():
    return RunnableParallel(
        account_info=RunnableLambda(get_account_info),
        order_history=RunnableLambda(get_order_history),
        ticket=RunnablePassthrough(),
    )
```

```
# main.py -- timing comparison
import time
from lookups import get_account_info, get_order_history
from research import build_research_chain

ticket_input = {"ticket": "..."}

# your turn: time calling get_account_info(ticket_input) then
# get_order_history(ticket_input) one after another, using
# time.perf_counter() before and after, and print the total.

# your turn: time calling build_research_chain().invoke(ticket_input)
# the same way, and print that total too.
```

Fill in both timing blocks yourself, run them, and compare your two printed numbers against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic is the plain plan with made-up 1-second delays. Intermediate is the real, runnable `lookups.py` and the shape of `build_research_chain()`. Advanced is the timing harness that turns "this should be faster" into a number you actually measured — which is the entire point of choosing `RunnableParallel` over just calling both functions in a row.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)